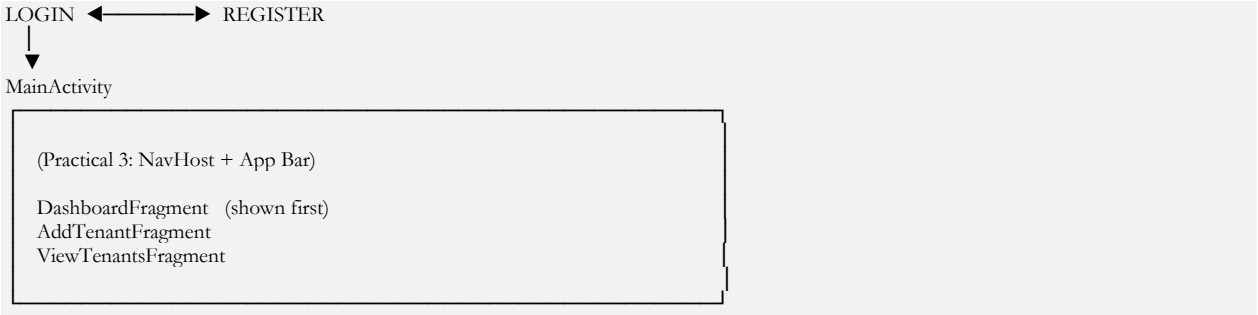
Strathmore
UNIVERSITY

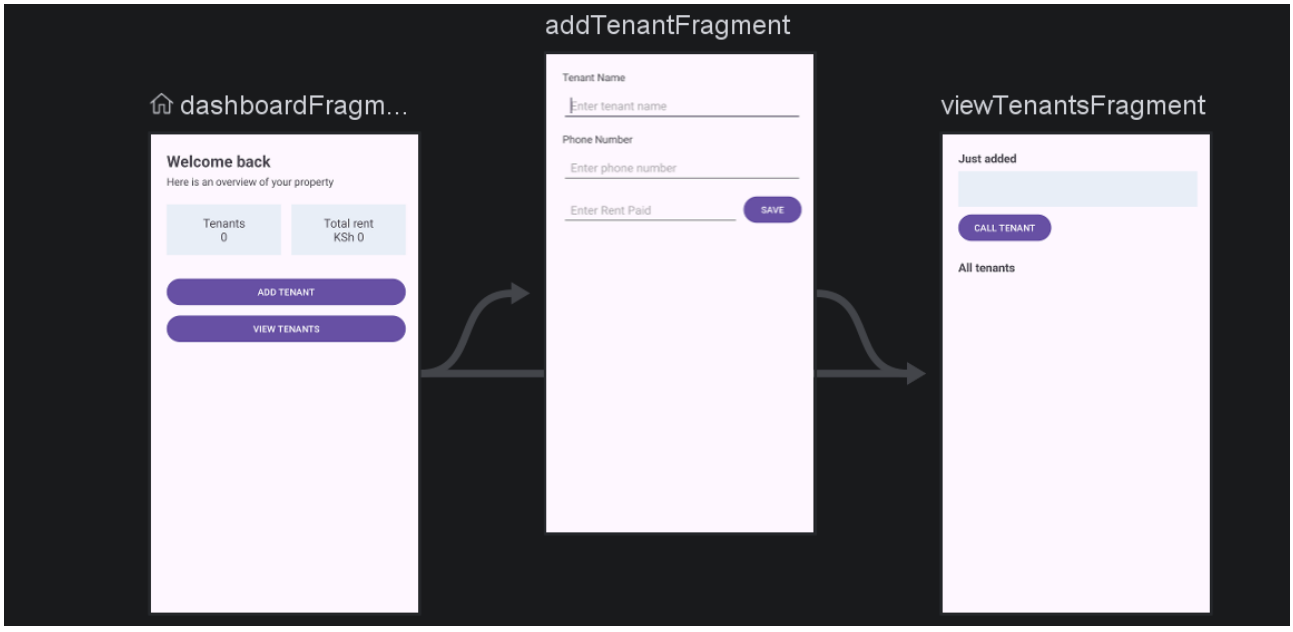
SCHOOL OF COMPUTING AND ENGINEERING SCIENCES
BACHELOR IN BUSINESS INFORMATION TECHNOLOGY
BBT 3.2: Mobile Application Development
Language: Kotlin
Lesson 6
Navigation Practical 2 — LifeCycles and Fragments
App Navigation, Part 2 of 3

What we are building

In Practical 1, you built Login and Register as separate Activities and moved between them with Intents. After logging in, the user landed on the Add Tenant screen.

In this practical, you first look **inside** an Activity to see how Android starts, pauses and destroys it (the **lifecycle**), and how to keep data safe when the screen rotates. Then you build the screens that will live inside MainActivity as **Fragments**, and map them in a **Navigation Graph**:





By the end, your three screens and the map connecting them are ready. In **Practical 3**, you'll plug them into MainActivity, move between them, pass data between them, and add a menu and a navigation drawer.

We will work through:

1. Watch the Activity lifecycle in Logcat
2. Save and restore data when the screen rotates

3. Set up the Navigation Component and Safe Args
4. Make Tenant Serializable and create a place to keep tenants
5. Create the Dashboard, Add Tenant and View Tenants Fragments
6. Build the Navigation Graph

This practical continues from **Practical 1**. You need Login and Register working, and the Add Tenant screen in MainActivity with the CALL TENANT button.

Throughout this guide, boxes marked **Question** answer the things students most often get stuck on. If something goes wrong, check the nearest Question box first, then the **Common errors** table at the end.

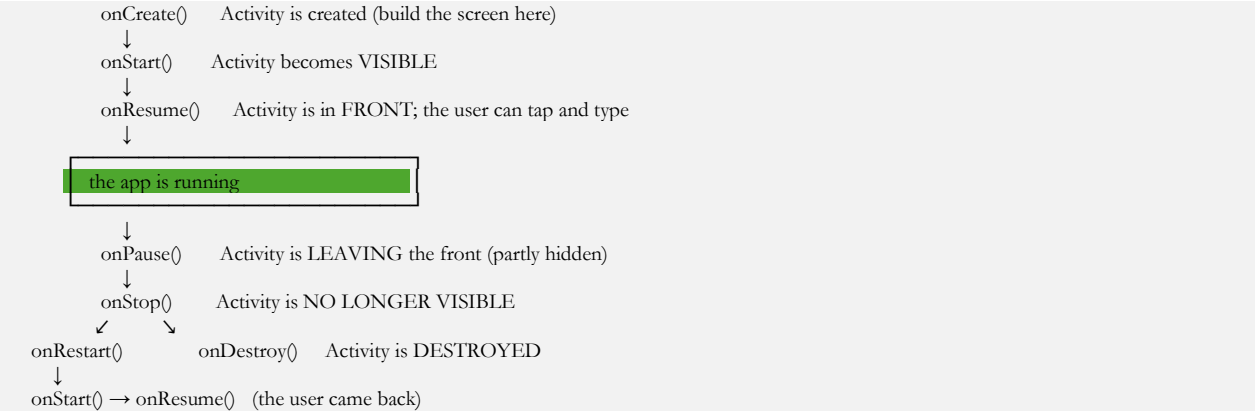
Everywhere this guide shows `com.example.tenantmanagementsystemgroupa`, use your own package name. You can find it on the first line of MainActivity.kt.

> **Note: Read the code, don't just copy and paste.**
> Every code block has comments (lines starting with // in Kotlin, or <!-- --> in XML) explaining what the code does and how the pieces connect. Read them before you paste, and make sure you can explain each line.

PART A — The Activity lifecycle

An Activity is not simply "open" or "closed". While your app runs, Android moves each Activity through several **states**: being created, becoming visible, being used, being covered by something else, and being destroyed. Each time the state changes, Android calls a **lifecycle method** in your Activity.

You already use one of them: onCreate(). There are six more.



Method	When Android calls it	Example of what you might do there
onCreate()	The Activity is created	Inflate the layout, set click listeners
onStart()	The screen becomes visible	Start showing data
onResume()	The user can now interact	Resume a paused video or game
onPause()	Something partly covers it, or the user is leaving	Pause a video
onStop()	The screen is fully hidden	Stop updates the user can't see
onRestart()	The user returns to a stopped Activity	Refresh data that may have changed
onDestroy()	The Activity is being removed	Final clean-up

We won't put real work in these methods today. We'll add a **log message** to each one, so we can watch Android call them as we use the app.

Step 1 — Log onCreate()

Open MainActivity.kt. At the top, with the other imports, add:

```
import android.util.Log
```

Inside onCreate(), directly **underneath** super.onCreate(savedInstanceState), add:

```
    // Write a message to Logcat every time onCreate() runs.
    // "TENANT_LIFECYCLE" is a tag: a label we search for in Logcat.
    Log.d("TENANT_LIFECYCLE", "MainActivity: onCreate")
```

> **Question: What is Log.d?**
> It writes a message to **Logcat**, Android Studio's log window. The user never sees it on the phone. d stands for "debug". The first value is the **tag** (a label for searching), and the second is the message.
>
> **Question: Why must super.onCreate(...) come first?**
> super runs Android's own code for that lifecycle step. Android expects it to run in every lifecycle method you override, or the app crashes with SuperNotCalledException.

Step 2 — Add the other lifecycle methods

Still in MainActivity.kt, scroll to the **end** of the class. Just above the very last } of the file (the one that closes class MainActivity), paste:

```
// ── LIFECYCLE METHODS ──────────────────────────────────────────
// Android calls each of these automatically. We only add a log
// line so we can watch the order in Logcat.

// The screen is now VISIBLE
override fun onStart() {
    super.onStart()
    Log.d("TENANT_LIFECYCLE", "MainActivity: onStart")
}

// The screen is in FRONT and the user can interact with it
override fun onResume() {
    super.onResume()
    Log.d("TENANT_LIFECYCLE", "MainActivity: onResume")
}

// The screen is LEAVING the front (e.g. Home was pressed)
override fun onPause() {
    super.onPause()
    Log.d("TENANT_LIFECYCLE", "MainActivity: onPause")
}

// The screen is NO LONGER VISIBLE
override fun onStop() {
    super.onStop()
    Log.d("TENANT_LIFECYCLE", "MainActivity: onStop")
}

// The user came BACK to a stopped screen
override fun onRestart() {
    super.onRestart()
    Log.d("TENANT_LIFECYCLE", "MainActivity: onRestart")
}

// The screen is being DESTROYED
override fun onDestroy() {
    super.onDestroy()
    Log.d("TENANT_LIFECYCLE", "MainActivity: onDestroy")
}
```

These methods go **inside** the class but **outside** onCreate():

```
class MainActivity : AppCompatActivity() {
    ├── onCreate() { ... }      ← existing code: binding, SAVE, CALL TENANT
    │
    ├── onStart()               ← new
    ├── onResume()              ← new
    ├── onPause()               ← new
    ├── onStop()                ← new
    ├── onRestart()             ← new
    └── onDestroy()             ← new
}
```

> **Question: Android Studio says "'onStart' overrides nothing".**
> The method is in the wrong place, probably inside onCreate(). Lifecycle methods must sit directly inside the class, at the same level as onCreate().
>
> **Question: Can I type these instead of pasting?**
> Yes. Inside the class, press **Ctrl+O** (Mac: **Cmd+O**), type the method name (for example onStart), and press Enter. Android Studio writes the method with the super call for you.

Step 3 — Watch the lifecycle in Logcat

1. Run the app.
2. At the bottom of Android Studio, click **Logcat**. (If you can't see it: **View → Tool Windows → Logcat**.)
3. In Logcat's search box, type:

```
tag:TENANT_LIFECYCLE
```

Now Logcat shows only our messages. Try each action below and compare what you see:

What you do	What Logcat shows
1. Log in (MainActivity opens)	onCreate → onStart → onResume
2. Press the phone's **Home** button	onPause → onStop
3. Open the app again from the recent apps	onRestart → onStart → onResume
4. Rotate the phone	onPause → onStop → onDestroy → onCreate → onStart → onResume
5. Press the phone's **Back** button	onPause → onStop → onDestroy

Look closely at action **4**. Rotating the phone **destroys MainActivity completely and creates a brand new one**. Android does this so the new Activity can load the right layout for the new screen shape.

> **Question: How do I rotate the emulator?**
> Use the two rotate buttons in the emulator's side toolbar. If the screen doesn't turn, swipe down from the top of the emulator screen and turn on **Auto-rotate**.
>
> **Question: Logcat shows nothing.**
> Check that the device selected at the top of Logcat is your running emulator, and that the search is exactly tag:TENANT_LIFECYCLE, with the same spelling as in your Log.d lines.
>
> **Question: In action 3, I saw onCreate instead of onRestart.**
> Android destroyed the stopped Activity to save memory while it was in the background. This is normal, especially on emulators with little memory. It's also why the next part matters.

Checkpoint. You can explain why each line appeared in Logcat for all five actions.

PART B — Saving State When the screen rotates

Step 4 — See the problem

1. Run the app and log in.
2. Enter a tenant and tap **SAVE**. The tenant's summary appears.
3. **Rotate** the phone.
4. Tap **CALL TENANT**.

You see *"Save a tenant first"*, even though you just saved one. The summary has also disappeared. But the text you typed in the three boxes is still there.

What happened:

```
Before rotating                    After rotating

MainActivity  (object 1)           MainActivity  (object 2, brand new)
├── lastTenant = John Kamau        ├── lastTenant = null
└── binding.tenant = John Kamau    └── binding.tenant = null
    │
    ✗  onDestroy(): object 1 and everything in it is gone
```

lastTenant and binding.tenant are variables **inside** the old Activity. When Android destroyed it, they were destroyed too. The new Activity starts from the beginning, with lastTenant = null.

> **Question: Then why did the text in the EditTexts survive?**
> Android automatically saves the text of every **EditText that has an android:id** and puts it back after rotating. It does not do this for your own Kotlin variables. Those you must save yourself.

Step 5 — Save the tenant in onSaveInstanceState()

Just before Android destroys an Activity it may recreate later, it calls onSaveInstanceState() and hands you a **Bundle**: a small box for saving values.

A Bundle works exactly like the **extras** on an Intent from Practical 1: you store each value under a **key**, and read it back later with the **same key**.

```
Practical 1 (Intent extras)          Today (Bundle)
intent.putExtra("EMAIL", email)      outState.putString("tenant_name", tenant.name)
intent.getStringExtra("EMAIL")       savedInstanceState.getString("tenant_name")
```

In MainActivity.kt, add this method inside the class, **above** your onStart() method:

```
// ── SAVE STATE ──────────────────────────────────────────────────────
// Android calls this just before destroying the Activity
// (for example, when the phone rotates).
// outState is the Bundle that Android keeps for us.
override fun onSaveInstanceState(outState: Bundle) {
    super.onSaveInstanceState(outState)

    Log.d("TENANT_LIFECYCLE", "MainActivity: onSaveInstanceState")

    // Copy into a val so Kotlin knows it can't change after the check
    val tenant = lastTenant

    if (tenant != null) {
        // Store each field under its own KEY.
        // onCreate() reads them back with the SAME keys.
        outState.putString("tenant_name", tenant.name)
        outState.putString("tenant_phone", tenant.phone)
        outState.putString("tenant_rent", tenant.rent)
    }
}
```

> **Question: Why save three Strings and not the whole Tenant?**
> A Bundle can only hold certain types, such as String, Int and Boolean. A Tenant object can't go in as it is yet. Later in this practical, you'll make Tenant **Serializable,** which lets a whole object go into a Bundle.

Step 6 — Restore the tenant in onCreate()

When Android creates the new Activity, it passes the same Bundle into onCreate() as **savedInstanceState**. You've seen this parameter in every onCreate() so far. Now we use it.

In onCreate(), directly **underneath** setContentView(binding.root), add:

```
    // ── RESTORE STATE ──────────────────────────────────────────────────────
    // savedInstanceState is the Bundle we filled in onSaveInstanceState().
    // It is null the FIRST time the Activity opens (nothing saved yet).
    if (savedInstanceState != null) {

        // Read each value back using the SAME keys we saved with
        val name = savedInstanceState.getString("tenant_name")
        val phone = savedInstanceState.getString("tenant_phone")
        val rent = savedInstanceState.getString("tenant_rent")

        // Only rebuild the tenant if all three were saved
        if (name != null && phone != null && rent != null) {

            val tenant = Tenant(name, phone, rent)

            // Put the tenant back in BOTH places it lived before rotating:
            lastTenant = tenant          // so CALL TENANT works again
            binding.tenant = tenant      // so the summary appears again (Data Binding)
        }
    }
```

Your complete MainActivity.kt should now be:

```
package com.example.tenantmanagementsystemgroupa

import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.util.Log
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.example.tenantmanagementsystemgroupa.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding

    // The most recently saved tenant. Lost on rotation unless we save it
    private var lastTenant: Tenant? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d("TENANT_LIFECYCLE", "MainActivity: onCreate")

        // Create the views from activity_main.xml and show them
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        // ── RESTORE STATE (only after a rotation) ──────────────────────────
        if (savedInstanceState != null) {
            val name = savedInstanceState.getString("tenant_name")
            val phone = savedInstanceState.getString("tenant_phone")
            val rent = savedInstanceState.getString("tenant_rent")

            if (name != null && phone != null && rent != null) {
                val tenant = Tenant(name, phone, rent)
                lastTenant = tenant
                binding.tenant = tenant
            }
        }

        // ── SAVE button ────────────────────────────────────────────────────
        binding.saveButton.setOnClickListener {
            val name = binding.tenantNameEditText.text.toString()
            val phone = binding.phoneEditText.text.toString()
            val rent = binding.rentEditText.text.toString()

            val tenant = Tenant(name, phone, rent)
            binding.tenant = tenant      // Data Binding shows the summary
            lastTenant = tenant          // remember it for CALL TENANT
        }

        // ── CALL TENANT button (implicit Intent) ───────────────────────────
```

```
        binding.callButton.setOnClickListener {
            val tenant = lastTenant

            if (tenant == null) {
                Toast.makeText(this, "Save a tenant first", Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }

            val intent = Intent(Intent.ACTION_DIAL, Uri.parse("tel:${tenant.phone}"))
            startActivity(intent)
        }
    }

    // ── SAVE STATE (just before the Activity is destroyed) ──────────
    override fun onSaveInstanceState(outState: Bundle) {
        super.onSaveInstanceState(outState)
        Log.d("TENANT_LIFECYCLE", "MainActivity: onSaveInstanceState")

        val tenant = lastTenant

        if (tenant != null) {
            outState.putString("tenant_name", tenant.name)
            outState.putString("tenant_phone", tenant.phone)
            outState.putString("tenant_rent", tenant.rent)
        }
    }

    // ── LIFECYCLE METHODS ──────────────────────────────────────────────
    override fun onStart() {
        super.onStart()
        Log.d("TENANT_LIFECYCLE", "MainActivity: onStart")
    }

    override fun onResume() {
        super.onResume()
        Log.d("TENANT_LIFECYCLE", "MainActivity: onResume")
    }

    override fun onPause() {
        super.onPause()
        Log.d("TENANT_LIFECYCLE", "MainActivity: onPause")
    }

    override fun onStop() {
        super.onStop()
        Log.d("TENANT_LIFECYCLE", "MainActivity: onStop")
    }

    override fun onRestart() {
        super.onRestart()
        Log.d("TENANT_LIFECYCLE", "MainActivity: onRestart")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d("TENANT_LIFECYCLE", "MainActivity: onDestroy")
    }
}
```

What happens when the phone rotates

```
OLD MainActivity                                    Logcat
─────────────────

onPause()                                           MainActivity: onPause
onStop()                                            MainActivity: onStop
onSaveInstanceState(outState)                       MainActivity: onSaveInstanceState
   outState.putString("tenant_name", "John Kamau")
   outState.putString("tenant_phone", "0712345678")
   outState.putString("tenant_rent", "25000")
       │
       │   Android keeps the Bundle while the old Activity is destroyed
       ▼
onDestroy()                                         MainActivity: onDestroy

NEW MainActivity
─────────────────

onCreate(savedInstanceState)                        MainActivity: onCreate
   savedInstanceState.getString("tenant_name") → "John Kamau"
   lastTenant = tenant
   binding.tenant = tenant  → summary appears again
onStart()                                            MainActivity: onStart
onResume()                                           MainActivity: onResume
```

Checkpoint. Run the app and repeat Step 4.

1. Save a tenant and rotate. The summary is still there.
2. Tap **CALL TENANT**. The dialer opens with the right number.
3. In Logcat, `onSaveInstanceState` appears between `onStop` and `onDestroy`.

What survives, and what doesn't:

Data	Survives rotation?	Why
Text typed in an EditText with an ID	Yes	Android saves it automatically
A Kotlin variable (e.g. `lastTenant`)	Only if you save it	It lives inside the destroyed Activity
Values saved in `onSaveInstanceState`	Yes	Android keeps the Bundle and gives it back
Anything, after pressing Back to close the screen	No	The user chose to leave, so Android doesn't keep the Bundle

Question: Why check `if (savedInstanceState != null)`?
The first time MainActivity opens (after Login), nothing has been saved, so `savedInstanceState` is `null`. It only contains our values after a rotation (or when Android recreates the Activity in the background).

Question: What is `onRestoreInstanceState()`?
Another place you could read the Bundle, called after `onStart()`. Reading it in `onCreate()` is enough for this practical, so you don't need both.

Question: Is the Bundle permanent storage?
No. It is for short interruptions like rotation. To keep tenants after the app is closed, you need a database. That comes later in the course.

From Activities to Fragments

You now have one Activity doing everything: the form, the result, and the CALL TENANT button. In the rest of this practical, MainActivity becomes a **frame**, and each screen inside it is a **Fragment**:

```
Activities (moved between with Intents — Practical 1)
├── LoginActivity
├── RegisterActivity
└── MainActivity
    └── NavHost (moved between with the Navigation Component — Practical 3)
        ├── DashboardFragment
        ├── AddTenantFragment
        └── ViewTenantsFragment
```

A **Fragment** is a reusable piece of UI that lives inside an Activity. Like an Activity, it has its own layout file, its own Kotlin class and its own lifecycle, but it cannot exist on its own.

```
Activity  = the whole window
Fragment  = a section inside it, which can be swapped for another
```

PART C — Set up the Navigation Component

From here on, we build the screens inside MainActivity with Fragments and the Navigation Component. First we add the libraries.

Step 7 — Add the Safe Args plugin to the Project Gradle file

This time we need **both** Gradle files. Open **build.gradle.kts (Project: TenantManagementSystem)**.

You will see something similar to:

```
plugins {
    alias(libs.plugins.android.application) apply false
}
```

Add one line inside the block:

```
plugins {
    alias(libs.plugins.android.application) apply false
    // Safe Args: generates classes for passing data between screens
    id("androidx.navigation.safeargs.kotlin") version "2.9.8" apply false
}
```

> **Question: Last time we were told never to open the Project file. Why now?**
> Most settings belong in the Module file. Safe Args is a Gradle plugin, and Gradle plugins with a version number have to be declared once at project level before an app module can use them. This is the only change we make here.
>
> **Question: What does `apply false` mean?**
> "Make this plugin available, but don't switch it on here." We switch it on in the app module in the next step.

Step 8 — Update the Module Gradle file

Open **build.gradle.kts (Module :app)**.

First, in the `plugins { }` block at the top, add two lines. It should look like:

```
plugins {
    alias(libs.plugins.android.application)
    // NEW: switch Safe Args on for this app.
    // It must come BELOW the android.application line
```

```
    id("androidx.navigation.safeargs.kotlin")
}
```

Then, scroll to the dependencies { } block add these two lines at the top of it:

```
implementation("androidx.navigation:navigation-fragment-ktx:2.9.8")
implementation("androidx.navigation:navigation-ui-ktx:2.9.8")
```

Click **Sync Now** and wait for Gradle to finish.

What each line does:

Line	What it gives you
androidx.navigation.safeargs.kotlin	Safe Args: generates classes for navigating and passing data safely
navigation-fragment-ktx	NavHostFragment and NavController
navigation-ui-ktx	NavigationUI: connects the App Bar and menus to navigation

> **Question: Do the version numbers have to match?**
> Yes. The Safe Args version in the Project file (Step 7) and the two navigation library versions here must all be **2.9.8.** If Android Studio suggests a newer version, update all three together.
>
> **Question: Sync failed with "safeargs plugin must be used with android plugin".**
> **Check three things:**
>
> 1. The version is 2.9.8 in both files. Older versions, such as 2.8.9, don't work with new projects.
> 2. In the Project file, the line ends with apply false.
> 3. In the Module file, the Safe Args line comes below alias(libs.plugins.android.application).

> **Question: Sync failed with "Plugin [id: 'androidx.navigation.safeargs.kotlin'] was not found".**
> You probably skipped Step 7, or added the line to the wrong file. The line with version "2.9.8" apply false goes in the Project file. The line without a version goes in the Module file.

PART D — Prepare the data

Step 9 — Make Tenant Serializable

To send a whole Tenant from one screen to another, Android needs to be able to pack it up and unpack it on the other side. This is called **serialization**. There are two ways to do it:

	Serializable	Parcelable
Comes from	Java/Kotlin itself	Android
Setup	Add : Serializable. Nothing else	Needs the kotlin-parcelize plugin, or a lot of hand-written code
Speed	Slower	Faster
Good for	Small objects, like one Tenant	Large objects or long lists sent often

For one small Tenant, the speed difference is too small to notice, so we use **Serializable**.

Open Tenant.kt and replace its contents with:

```
package com.example.tenantmanagementsystemgroupa
```

```
// Serializable is built into Java/Kotlin: no plugin needed
import java.io.Serializable

// ": Serializable" means "a Tenant can be packed up and sent
// to another screen". Safe Args needs this in Practical 3
data class Tenant(
    val name: String,
    val phone: String,
    val rent: String
) : Serializable {

    // Unchanged: the text shown by @{tenant.summary()}
    fun summary(): String {
        return "Tenant: $name\nPhone: $phone\nRent: KSh $rent"
    }
}
```

Only two things changed: the import java.io.Serializable line, and : Serializable after the class.

> **Question: The slides say Parcelable is faster. Why don't we use it?**
> Parcelable needs the kotlin-parcelize plugin, which doesn't work with the built-in Kotlin in new Android Studio projects. Writing Parcelable by hand takes about 30 extra lines of code. For one small Tenant, Serializable works just as well and needs no setup.

Step 10 — Create a place to keep tenants: TenantRepository

Right-click com.example.tenantmanagementsystemgroupa, choose **New → Kotlin Class/File**, type:

```
TenantRepository
```

This time choose **Object** (not Class) and press Enter.

Replace the contents with:

```
package com.example.tenantmanagementsystemgroupa

object TenantRepository {

    val tenants = mutableListOf<Tenant>()

    fun addTenant(tenant: Tenant) {
        tenants.add(tenant)
    }

    fun totalRent(): Double {
        return tenants.sumOf { it.rent.toDoubleOrNull() ?: 0.0 }
    }
}
```

> **Question: What is an object?**
> A class that has exactly **one** instance, shared by the whole app. The Dashboard, Add Tenant and View Tenants screens all see the same list of tenants.
>
> **Question: What does it.rent.toDoubleOrNull() ?: 0.0 do?**
> It converts each tenant's rent from text to a number. If the text isn't a valid number, it uses 0.0 instead of crashing.
>
> **Question: Will my tenants still be there if I close the app?**
> No. The list lives in memory only. Saving data permanently is a later topic.

PART E — Create the Fragments

Each screen inside MainActivity is a **Fragment**: a layout file plus a Kotlin class. We create three.

Step 11 — Create the three layout files

For each file below: right-click the **layout** folder, choose **New → Layout Resource File**, type the file name, set the root element to androidx.constraintlayout.widget.ConstraintLayout, and click **OK**.

```
fragment_dashboard
fragment_add_tenant
fragment_view_tenants
```

You should now have:

```
res
└── layout
    ├── activity_login.xml
    ├── activity_main.xml
    ├── activity_register.xml
    ├── fragment_add_tenant.xml
    ├── fragment_dashboard.xml
    └── fragment_view_tenants.xml
```

Step 12 — Design fragment_dashboard.xml

Open fragment_dashboard.xml, click **Code**, delete everything and paste:

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="24dp">

    <TextView
        android:id="@+id/welcomeTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome back"
        android:textSize="24sp"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent" />

    <TextView
        android:id="@+id/overviewTextView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Here is an overview of your property"
        android:textSize="16sp"
        android:layout_marginTop="4dp"
        app:layout_constraintTop_toBottomOf="@id/welcomeTextView"
        app:layout_constraintStart_toStartOf="parent" />

    <TextView
        android:id="@+id/tenantCountTextView"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:text="Tenants\n0"
        android:textSize="18sp"
        android:gravity="center"
        android:padding="16dp"
        android:background="#E8EEF7"
        android:layout_marginTop="24dp"
        android:layout_marginEnd="8dp"
        app:layout_constraintTop_toBottomOf="@id/overviewTextView"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toStartOf="@id/totalRentTextView" />

    <TextView
        android:id="@+id/totalRentTextView"
```

```
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:text="Total rent\nKSh 0"
    android:textSize="18sp"
    android:gravity="center"
    android:padding="16dp"
    android:background="#E8EEF7"
    android:layout_marginStart="8dp"
    app:layout_constraintTop_toTopOf="@id/tenantCountTextView"
    app:layout_constraintStart_toEndOf="@id/tenantCountTextView"
    app:layout_constraintEnd_toEndOf="parent" />

  <Button
    android:id="@+id/addTenantButton"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:text="ADD TENANT"
    android:layout_marginTop="32dp"
    app:layout_constraintTop_toBottomOf="@id/tenantCountTextView"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />

  <Button
    android:id="@+id/viewTenantsButton"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:text="VIEW TENANTS"
    android:layout_marginTop="8dp"
    app:layout_constraintTop_toBottomOf="@id/addTenantButton"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

The two metric boxes are a **horizontal chain** from the ConstraintLayout lesson: each is constrained to the other, and both have 0dp width, so they share the row equally.

Step 13 — Design fragment_add_tenant.xml

This is the form from the last practical, moved into its own screen. The title is gone because the App Bar will show "Add Tenant", and the result text is gone because saving now takes you to the View Tenants screen.

Open fragment_add_tenant.xml, click **Code**, delete everything and paste:

```
<?xml version="1.0" encoding="utf-8"?>

<androidx.constraintlayout.widget.ConstraintLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  android:layout_width="match_parent"
  android:layout_height="match_parent">

  <TextView
    android:id="@+id/tenantNameLabel"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Tenant Name"
    android:textSize="16sp"
    android:layout_marginTop="24dp"
    android:layout_marginStart="24dp"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintStart_toStartOf="parent" />

  <EditText
    android:id="@+id/tenantNameEditText"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:hint="Enter tenant name"
    android:minHeight="48dp"
    android:padding="12dp"
    android:layout_marginTop="8dp"
    android:layout_marginStart="24dp"
    android:layout_marginEnd="24dp"
    app:layout_constraintTop_toBottomOf="@id/tenantNameLabel"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

```
    <TextView
        android:id="@+id/phoneLabel"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Phone Number"
        android:textSize="16sp"
        android:layout_marginTop="16dp"
        android:layout_marginStart="24dp"
        app:layout_constraintTop_toBottomOf="@id/tenantNameEditText"
        app:layout_constraintStart_toStartOf="parent" />

    <EditText
        android:id="@+id/phoneEditText"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="Enter phone number"
        android:inputType="phone"
        android:minHeight="48dp"
        android:padding="12dp"
        android:layout_marginTop="8dp"
        android:layout_marginStart="24dp"
        android:layout_marginEnd="24dp"
        app:layout_constraintTop_toBottomOf="@id/phoneLabel"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

    <EditText
        android:id="@+id/rentEditText"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="Enter Rent Paid"
        android:inputType="numberDecimal"
        android:minHeight="48dp"
        android:padding="12dp"
        android:layout_marginStart="24dp"
        android:layout_marginTop="16dp"
        android:layout_marginEnd="8dp"
        app:layout_constraintTop_toBottomOf="@id/phoneEditText"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toStartOf="@id/saveButton" />

    <Button
        android:id="@+id/saveButton"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="SAVE"
        android:layout_marginEnd="24dp"
        app:layout_constraintTop_toTopOf="@id/rentEditText"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Step 14 — Design fragment_view_tenants.xml (with Data Binding)

This screen shows the tenant that was just added, using Data Binding exactly as in the last practical, and then a list of all tenants.

Open fragment_view_tenants.xml, click **Code**, delete everything and paste:

```
<?xml version="1.0" encoding="utf-8"?>
<layout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">

    <data>
        <variable
            name="tenant"
            type="com.example.tenantmanagementsystemgroupa.Tenant" />
    </data>

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <androidx.constraintlayout.widget.ConstraintLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="24dp">
```

```
        <TextView
            android:id="@+id/newTenantLabel"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Just added"
            android:textSize="18sp"
            android:textStyle="bold"
            app:layout_constraintTop_toTopOf="parent"
            app:layout_constraintStart_toStartOf="parent" />

        <TextView
            android:id="@+id/newTenantTextView"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:text="@{tenant.summary()}"
            android:textSize="16sp"
            android:padding="16dp"
            android:background="#E8EEF7"
            android:layout_marginTop="8dp"
            app:layout_constraintTop_toBottomOf="@id/newTenantLabel"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toEndOf="parent" />

        <Button
            android:id="@+id/callButton"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="CALL TENANT"
            android:layout_marginTop="8dp"
            app:layout_constraintTop_toBottomOf="@id/newTenantTextView"
            app:layout_constraintStart_toStartOf="parent" />

        <androidx.constraintlayout.widget.Group
            android:id="@+id/newTenantGroup"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            app:constraint_referenced_ids="newTenantLabel,newTenantTextView,callButton" />

        <TextView
            android:id="@+id/allTenantsLabel"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="All tenants"
            android:textSize="18sp"
            android:textStyle="bold"
            android:layout_marginTop="24dp"
            app:layout_goneMarginTop="0dp"
            app:layout_constraintTop_toBottomOf="@id/callButton"
            app:layout_constraintStart_toStartOf="parent" />

        <TextView
            android:id="@+id/allTenantsTextView"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:textSize="16sp"
            android:lineSpacingExtra="4dp"
            android:layout_marginTop="8dp"
            app:layout_constraintTop_toBottomOf="@id/allTenantsLabel"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toEndOf="parent" />

    </androidx.constraintlayout.widget.ConstraintLayout>
  </ScrollView>
</layout>
```

Two ConstraintLayout features from the earlier lesson appear here:

- The **Group** lets us hide "Just added", the tenant box and the Call button together with one line of code, for example when we open this screen from the Dashboard and no tenant was just added.
- **goneMarginTop** makes "All tenants" move to the top without a gap when that group is hidden.

> **Question: Why a ScrollView here?**
> The list of all tenants can grow longer than the screen. The ScrollView lets the user scroll down to see it all.

> **Question: Should I change the package in type="..."?**
> Yes, if your package is different. It must be your package name followed by .Tenant.

Step 15 — Create the Fragment classes

For each class below: right-click com.example.tenantmanagementsystemgroupa, choose **New → Kotlin Class/File**, type the name, choose **Class**, and press Enter.

```
DashboardFragment
AddTenantFragment
ViewTenantsFragment
```

> **Question: Why not use "New → Fragment → Fragment (Blank)"?**
> That wizard adds a lot of extra code we don't need (like ARG_PARAM1 and newInstance). Writing the class ourselves keeps it short and easy to follow.

Step 16 — Write DashboardFragment.kt

Open DashboardFragment.kt. Delete everything **except the package line**, then paste:

```
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
// Generated from fragment_dashboard.xml (View Binding)
import com.example.tenantmanagementsystemgroupa.databinding.FragmentDashboardBinding

class DashboardFragment : Fragment() {

    // _binding can be null (no view yet, or view destroyed)
    private var _binding: FragmentDashboardBinding? = null
    // binding is a shortcut we only use while the view exists
    private val binding get() = _binding!!

    // Build this screen's view from fragment_dashboard.xml
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentDashboardBinding.inflate(inflater, container, false)
        return binding.root   // hand the view to the NavHost to display
    }

    // The view now exists: fill in the metrics
    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        // Read from the shared list of tenants
        val tenantCount = TenantRepository.tenants.size

        val totalRent = TenantRepository.totalRent()

        // tenantCountTextView ← android:id="@+id/tenantCountTextView"
        binding.tenantCountTextView.text = "Tenants\n$tenantCount"

        // totalRentTextView ← android:id="@+id/totalRentTextView"
        binding.totalRentTextView.text = "Total rent\nKSh %,.0f".format(totalRent)
    }

    // The view is destroyed: let go of it
    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

View Binding in a Fragment looks a little different from an Activity, because a Fragment's view can be destroyed and recreated while the Fragment itself stays alive.

```
Fragment lifecycle (the parts we use)

onCreateView()   →  build the view from XML (create the binding)
     ↓
onViewCreated()  →  the view exists: set text, add click listeners
     ↓
onDestroyView()  →  the view is gone: clear the binding
```

Question: Why two variables, _binding and binding?
_binding can be null (before the view is created and after it is destroyed). binding is a shortcut that says "I know it's not null right now", so we can write binding.saveButton instead of _binding?.saveButton everywhere. Only use binding between onCreateView and onDestroyView.

Question: Why set _binding = null in onDestroyView()?
When you navigate away, the Fragment's view is destroyed but the Fragment may be kept for when you come back. Holding on to the old view would waste memory. Setting it to null lets Android clean it up.

Question: Why inflate(inflater, container, false) here but inflate(layoutInflater) in an Activity?
A Fragment's view is placed inside a container (the NavHost). We pass the container so the layout sizes itself correctly, and false because the NavHost attaches the view itself.

Question: What does "KSh %,.0f".format(totalRent) do?
It formats the number with commas and no decimals, so 75000.0 becomes KSh 75,000.

Step 17 — Write AddTenantFragment.kt

Open AddTenantFragment.kt. Delete everything **except the package line**, then paste:

```
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import com.example.tenantmanagementsystemgroupa.databinding.FragmentAddTenantBinding

class AddTenantFragment : Fragment() {

    private var _binding: FragmentAddTenantBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentAddTenantBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        // The SAVE button is added in Practical 3
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Step 18 — Write ViewTenantsFragment.kt

Open ViewTenantsFragment.kt. Delete everything **except the package line**, then paste:

```
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import com.example.tenantmanagementsystemgroupa.databinding.FragmentViewTenantsBinding

class ViewTenantsFragment : Fragment() {
```

```
    private var _binding: FragmentViewTenantsBinding? = null
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        _binding = FragmentViewTenantsBinding.inflate(inflater, container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        // For now, hide the "Just added" section. Practical 3 shows it when a tenant is passed in.
        binding.newTenantGroup.visibility = View.GONE

        val tenants = TenantRepository.tenants

        if (tenants.isEmpty()) {
            binding.allTenantsTextView.text = "No tenants yet. Add one from the Dashboard."
        } else {
            binding.allTenantsTextView.text = tenants.joinToString("\n\n") { it.summary() }
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

> **Question: What does joinToString("\n\n") { it.summary() } do?**
> It takes every tenant, turns each into its summary text, and joins them together with a blank line between each one.
>
> **Question: Will this list become a RecyclerView?**
> Yes. For now, a TextView is enough to see the tenants. We replace it with a RecyclerView when we cover lists.

PART F — Build the Navigation Graph

The **Navigation Graph** is a map of all the screens (destinations) in MainActivity and the paths (actions) between them.

Step 19 — Create nav_graph.xml

Right-click the **res** folder, then choose:

```
New → Android Resource File
```

Fill in the window:

```
File name:      nav_graph
Resource type:  Navigation
```

Click **OK**. Android Studio creates a new folder:

```
res
└── navigation
    └── nav_graph.xml
```

> **Question: I don't see "Navigation" in the Resource type list.**
> The navigation libraries from Step 8 are missing or not synced. Sync and try again.

Step 20 — Add the destinations and actions

Open nav_graph.xml, click **Code**, delete everything and paste:

```
<?xml version="1.0" encoding="utf-8"?>
<navigation xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/nav_graph"
    app:startDestination="@id/dashboardFragment">

    <fragment
        android:id="@+id/dashboardFragment"
        android:name="com.example.tenantmanagementsystemgroupa.DashboardFragment"
        android:label="Dashboard"
        tools:layout="@layout/fragment_dashboard">

        <action
            android:id="@+id/action_dashboardFragment_to_addTenantFragment"
            app:destination="@id/addTenantFragment" />

        <action
            android:id="@+id/action_dashboardFragment_to_viewTenantsFragment"
            app:destination="@id/viewTenantsFragment" />
    </fragment>

    <fragment
        android:id="@+id/addTenantFragment"
        android:name="com.example.tenantmanagementsystemgroupa.AddTenantFragment"
        android:label="Add Tenant"
        tools:layout="@layout/fragment_add_tenant">

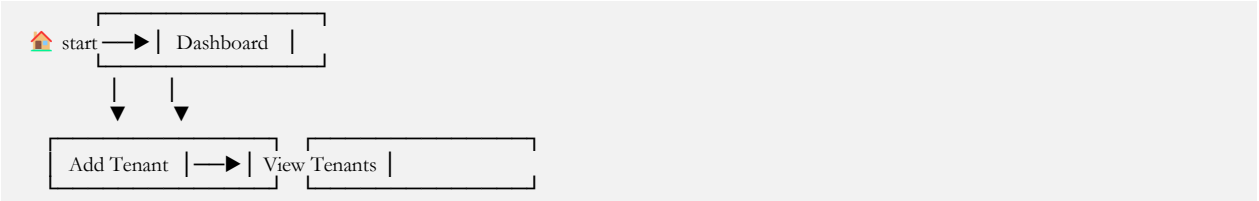
        <action
            android:id="@+id/action_addTenantFragment_to_viewTenantsFragment"
            app:destination="@id/viewTenantsFragment" />
    </fragment>

    <fragment
        android:id="@+id/viewTenantsFragment"
        android:name="com.example.tenantmanagementsystemgroupa.ViewTenantsFragment"
        android:label="View Tenants"
        tools:layout="@layout/fragment_view_tenants">

        <argument
            android:name="newTenant"
            app:argType="com.example.tenantmanagementsystemgroupa.Tenant"
            app:nullable="true"
            android:defaultValue="@null" />
    </fragment>

</navigation>
```

Now click **Design** in the top-right corner. This is the **Navigation Editor**. You should see the three screens with arrows between them:



The parts of the graph:

Part	What it means
app:startDestination	The first screen shown inside MainActivity (the house icon in the editor)
<fragment>	One destination (screen)
android:name	The Kotlin class for that screen

Part	What it means
android:label	The title shown in the App Bar on that screen
<action>	An arrow: a path from one screen to another
<argument>	Data the destination expects to receive

> **Question: Could I draw the arrows in the Design view instead of typing XML?**
> Yes. In Design view, hover over a screen, drag from the small circle on its right edge to another screen, and Android Studio writes the <action> for you. The action IDs follow the pattern action_<from>_to_<to>, which is exactly what we typed.
>
> **Question: Why is the newTenant argument nullable with a default of @null?**
> View Tenants can be opened two ways. From Add Tenant, we send the tenant that was just saved. From the Dashboard, there is no new tenant to send. nullable="true" and defaultValue="@null" mean "sending a tenant is optional".
>
> **Question: Why can we use Tenant as an argument type?**
> Because we made it Serializable in Step 9. Only simple types (like String, Int, Boolean) and Parcelable or Serializable classes can be passed as arguments.
>
> **Question: The design view shows "No preview" for a screen.**
> Check the tools:layout line for that destination. It only affects the preview, not the app.

Checkpoint. Run the app and log in. MainActivity still shows the Add Tenant form from Practical 1. That's expected: the Fragments aren't shown anywhere yet. Open nav_graph.xml in **Design** view and check that you can see the three screens and the arrows between them. In Practical 3, you'll plug them into MainActivity.

Final project structure

```
TenantManagementSystem
└── app
    ├── kotlin+java
    │   └── com.example.tenantmanagementsystemgroupa
    │       ├── LoginActivity.kt
    │       ├── RegisterActivity.kt
    │       ├── MainActivity.kt
    │       ├── DashboardFragment.kt
    │       ├── AddTenantFragment.kt
    │       ├── ViewTenantsFragment.kt
    │       ├── Tenant.kt
    │       └── TenantRepository.kt
    └── res
        ├── layout
        │   ├── activity_login.xml
        │   ├── activity_register.xml
        │   ├── activity_main.xml
        │   ├── fragment_dashboard.xml
        │   ├── fragment_add_tenant.xml
        │   └── fragment_view_tenants.xml
        └── navigation
            └── nav_graph.xml
```

Where things stand at the end of this practical:

```
LoginActivity
    │ explicit Intent
    ▼
MainActivity  (Add Tenant form, lifecycle logs, saved state)

nav_graph.xml  (ready for Practical 3)
    ├── DashboardFragment    (start destination)
    ├── AddTenantFragment
    └── ViewTenantsFragment
```

Common errors

What you see	Cause	Fix
Logcat shows none of our messages	Wrong device selected, or search doesn't match the tag	Select your emulator; search tag:TENANT_LIFECYCLE
'onStart' overrides nothing	Lifecycle method placed inside onCreate()	Move it to class level (Step 2)
SuperNotCalledException	super.onStart() (etc.) missing	Call super first in every lifecycle method
Screen doesn't rotate	Auto-rotate is off on the emulator	Turn on **Auto-rotate** in quick settings
CALL TENANT says "Save a tenant first" after rotating	State not saved and restored	Add Steps 5 and 6
Plugin ... safeargs ... was not found	Missing line in the Project Gradle file	Add it (Step 7) and sync
Parcelize is red	kotlin-parcelize plugin missing	Add it to the Module Gradle file (Step 8)
"Navigation" missing from Resource type	Navigation libraries not added or not synced	Step 8, then sync
NullPointerException on binding in a Fragment	Used binding after onDestroyView or before onCreateView	Only use binding inside onViewCreated and click listeners

---

Try it yourself

1. **Lifecycle of Login.** Add lifecycle logs to LoginActivity (use the message "LoginActivity: onCreate" and so on). Log in and watch the order. When does LoginActivity get onDestroy compared with MainActivity's onCreate? (Hint: finish().)
2. **Lifecycle of Register.** Add lifecycle logs to RegisterActivity too. Open Register and tap **Already have an account? Log in**. Which methods run, and why does Register get onDestroy?
3. **Explore the Navigation Editor.** Open nav_graph.xml in Design view. Drag a new arrow from View Tenants back to Dashboard, then look at the <action> Android Studio wrote in Code view. Delete it again when you're done.
4. **Save something else.** In Part B, also save how many times SAVE has been tapped (an Int), using outState.putInt(...) and savedInstanceState.getInt(..., 0). Show the count in a Toast after rotating.

What you learned today

```
STEP 1   Activity lifecycle
       "I know when Android creates, pauses, stops and destroys my screen."
             ↓
STEP 2   Saving state
       "I can keep data safe when the screen rotates."
             ↓
STEP 3   Fragments
       "I can build screens that live inside one Activity."
             ↓
STEP 4   Navigation Graph
       "I can map my screens and the paths between them."
```

Coming up next

In **Practical 3**, you show the Fragments inside MainActivity with a NavHost and App Bar, make the Dashboard buttons move between screens, send a tenant from Add Tenant to View Tenants with **Safe Args**, watch the **Fragment lifecycle**, and add an **options menu** and a **navigation drawer**.